Project Report: Vibe Coding Final Project

Project Name: CareerPilot AI **Team Name:** Wolf **Team Members:** Anna Palannatt, Joel George **Live Application URL:** https://careerpilot-ai-plum.vercel.app/ **GitHub Repository:** https://github.com/joelgeorge05/careerpilot-ai

---

1. Application Overview and Tech Stack

Overview CareerPilot AI is an AI-powered career discovery and analysis platform designed to help professionals and job seekers navigate their career paths, tailor their resumes, analyze job descriptions, and prepare for interviews. The platform is built around four core modules: Career Discovery, Resume Analyzer, Job Description (JD) Analyzer, and Interview Coach.

Tech Stack - **Frontend:** React, Vite, Tailwind CSS - **Backend:** Python, FastAPI - **AI Model:** Google Gemini API (via the `google-generativeai` SDK) - **Deployment:** Docker, Amazon Web Services (AWS)

---

2. Prompting Strategy and Frameworks Used

To develop the application rapidly using Vibe Coding, I heavily relied on **zero-shot** and **few-shot prompting**, alongside **role-prompting frameworks**.

Role-Prompting

For each module in the application, the Gemini model was prompted to take on a highly specific persona to ensure output consistency and professionalism. * **Sample Prompt Fragment:** *"You are an elite career coach and resume expert with 15 years of experience in technical recruiting. Your task is to analyze the provided resume against the target role."*

Structured Output Constraints

To ensure the LLM returned responses that could be reliably parsed and displayed progressively in the UI, prompts explicitly mandated formatting rules (like using markdown, bullet points, and specific headings). * **Sample Prompt (JD Analyzer):** *"Analyze the following job description. Output your analysis strictly using the following Markdown sections: 1. Core Responsibilities, 2. Required Technical Skills, 3. Soft Skills, 4. Experience Level, 5. Red Flags/Hidden Requirements. Do not output any preamble."*

---

3. Phase-by-Phase Development Summary

Phase 1: Conceptualization and Prototyping The initial phase involved brainstorming the four core modules. Using AI code generation tools, the wireframes and basic UI components were drafted in React with Tailwind CSS, focusing on a clean, modern, and dark-themed aesthetic.

Phase 2: Backend Architecture & AI Integration The FastAPI backend was initialized. Endpoints were created for each module (e.g., `/api/resume/analyze`, `/api/interview/questions`). The Google Gemini API was integrated, focusing specifically on enabling **streaming responses**. This allowed long AI responses to be streamed chunk-by-chunk to the frontend, drastically improving perceived performance.

Phase 3: Connecting the Stack The frontend was wired to the backend API. React state management and asynchronous streaming readers were implemented to handle the progressively loading text. CORS middleware was configured on the FastAPI server to allow cross-origin requests from the React development server.

Phase 4: Containerization and Deployment Preparation A `Dockerfile` was authored to containerize the Python backend. The application's environment variables (`GEMINI_API_KEY`, `FRONTEND_ORIGIN`) were abstracted to `.env` files to ensure sensitive keys were kept entirely out of version control, fulfilling the security requirements.

---

4. Application Architecture

The application follows a standard decoupled client-server architecture:

1. **Client Tier (React/Vite):** A single-page application (SPA) that manages user inputs, file uploads (for resumes), and UI state. It communicates with the backend via HTTP POST requests using the native Fetch API.
2. **API Tier (FastAPI):** A Python-based REST API that receives client payloads, structures them into engineered prompts, and manages the communication layer with the external LLM provider.
3. **LLM Tier (Google Gemini):** Processes the prompts and returns the analytical output as a continuous stream back to the FastAPI server, which is then piped directly back to the client.
4. **Container & Cloud Layer:** The backend service is encapsulated within a Docker container, ensuring environmental consistency, making it ready to be deployed to an AWS service (such as AWS App Runner).

---

5. Challenges Encountered and How They Were Resolved

Challenge 1: Streaming Long AI Responses * **Issue:** The AI model often took several seconds to generate complete resume or JD analyses. Waiting for the full response led to a poor user experience. * **Resolution:** I implemented a streaming architecture. On the backend, FastAPI's `StreamingResponse` was used in conjunction with Gemini's streaming capabilities. On the frontend, a custom text decoder was built to parse the response body stream in real-time, displaying characters as they arrived.

Challenge 2: Multi-part Form Data for Resumes * **Issue:** Sending PDF files alongside text parameters (like the target role) required moving away from standard JSON payloads. * **Resolution:** Implemented `python-multipart` on the FastAPI backend and refactored the frontend API call to use standard `FormData`, allowing the seamless upload and parsing of the user's PDF resume using `pypdf`.

Challenge 3: Dependency Version Conflicts * **Issue:** During setup, specific pinned versions of Python packages (like `pydantic-core`) lacked pre-compiled wheels for Python 3.13, causing build failures. * **Resolution:** I relaxed the strict version pinning in `requirements.txt`, allowing the package manager (`pip`) to automatically resolve and pull the latest versions compatible with the current Python environment.

---

6. Key Learnings and Reflection

This project demonstrated the immense power of **Vibe Coding**—the ability to act as a creative director and architect rather than solely focusing on syntax. By leveraging AI to write the boilerplate, styling, and standard logic, I was able to dedicate my time to the higher-level architecture: structuring the streaming endpoints, refining the prompts, and ensuring security best practices.

I learned that the key to building good AI applications lies not just in writing code, but in **Prompt Engineering**. The quality of the application is directly proportional to how well the backend constructs and constrains the context given to the LLM. Furthermore, containerizing the application using Docker underscored the importance of DevOps in the modern development lifecycle, ensuring that the jump from "working on localhost" to "deployed on AWS" is frictionless.